



Universidade Federal do Rio Grande do Norte

Centro de Tecnologia - CT

Departamento de Engenharia Elétrica - DEE

Laboratório de Automação, Controle e Instrumentação - LACI

Relatório Técnico

ELE3717 - Relatório 02

LÍVIA STEFANNY DE MOURA FELIPE
RUTILENO GABRIEL CAMILO DA SILVA

Natal, 30 de junho de 2025

RESUMO

Este relatório técnico documenta o desenvolvimento de *firmware* em linguagem C para o microcontrolador AVR Atmega328P, focando na implementação de sistemas embarcados interativos e de processamento de sinais. O trabalho aborda três projetos principais que demonstram a aplicação prática de conceitos de *hardware* e *software*: (1) um controlador de LED RGB com Interface Homem-Máquina (IHM) para ajuste de cores e gravação em memória não volátil; (2) uma versão do jogo *Snake* utilizando um *joystick* analógico para controle e uma matriz de LEDs 8x8 para visualização; e (3) um filtro digital FIR (*Finite Impulse Response*) com ajuste dinâmico dos coeficientes em tempo real.

A metodologia envolveu programação em C padrão ANSI com manipulação direta de registradores, utilizando periféricos como temporizadores/contadores para geração de PWM, conversor analógico-digital (ADC) para leitura de entradas, comunicação SPI para controle de drivers de *display* e acesso à memória EEPROM. A plataforma Arduino Uno foi utilizada para a prototipagem e validação dos sistemas.

Os resultados demonstram a implementação bem-sucedida das funcionalidades projetadas, incluindo a criação de interfaces responsivas, a gestão de lógica de jogo em tempo real e a aplicação de algoritmos de processamento digital de sinais. O desenvolvimento evidenciou a eficácia da linguagem C para gerenciar a complexidade e a importância do entendimento da arquitetura do hardware para solucionar desafios práticos, como a sincronização de periféricos e a garantia de temporização adequada.

Palavras-chave: Microcontroladores, ATmega328P, Linguagem C, Sistemas Embarcados, IHM, PWM, Jogo Embarcado, Filtro FIR, Processamento Digital de Sinais (PDS), EEPROM.

SUMÁRIO

1. INTRODUÇÃO.....	4
2. FUNDAMENTAÇÃO TEÓRICA.....	5
2.1 Microcontroladores e a Arquitetura AVR.....	5
2.2 ATmega328P.....	5
2.3 Programação em Linguagem C.....	6
2.4 Portas de Entrada/Saída (I/O) do ATmega328P.....	6
2.5 Matriz de LED e o Circuito Integrado MAX7219.....	7
2.6 Leitura de Botões/Teclas.....	8
2.7 Joystick.....	8
2.8 Temporizadores/Contadores (TCs).....	9
2.9 Memória EEPROM.....	9
3. MATERIAIS E MÉTODOS.....	10
3.1 Projeto 04 - Controle de LED RGB.....	10
Figura 01 - Máquina de estados para o algoritmo de funcionamento do Controle de LED RGB.....	11
3.1.1. Correções do Projeto 04.....	11
Figura 02 - Funcionamento do Controle de LED RGB.....	13
3.2 Projeto 05 - Jogo Snake Simplificado.....	13
3.2.1 Correções do Projeto 05.....	14
3.3 Projeto 06 - Filtro FIR com ajuste dinâmico.....	14
3.3.1 Correções do Projeto 06.....	15
4. RESULTADOS.....	15
Figura 03 - Sinal de entrada (amarelo) e sinal de saída (azul) do Filtro FIR projetado.....	16
5. CONCLUSÃO.....	17
REFERÊNCIAS BIBLIOGRÁFICAS.....	18
ANEXOS.....	19

Figuras

Figura 01 - Máquina de estados para o algoritmo de funcionamento do Controle de LED RGB.....	11
Figura 02 - Funcionamento do Controle de LED RGB.....	13
Figura 03 - Sinal de entrada (amarelo) e sinal de saída (azul) do Filtro FIR projetado..	16

1. INTRODUÇÃO

A contínua expansão da área de sistemas embarcados impulsiona a busca por soluções cada vez mais sofisticadas e eficientes. Neste cenário, microcontroladores como o AVR ATMega328 são ferramentas essenciais. Este relatório apresenta uma evolução metodológica no desenvolvimento de *firmware* para este microcontrolador, documentando a transição da programação em *Assembly* para a poderosa e flexível linguagem ANSI C. Essa mudança para um nível de abstração superior permitiu a implementação de projetos com maior complexidade lógica e interativa.

Para ilustrar essa capacidade, o trabalho detalha a construção de sistemas que exploram diferentes domínios da computação embarcada. As aplicações desenvolvidas demonstram a versatilidade do C em tarefas que abrangem desde a criação de interfaces homem-máquina (IHM) interativas com persistência de dados, passando pela gestão de lógica de jogos em tempo real, até a execução de algoritmos de processamento digital de sinais (PDS).

O objetivo central é documentar o processo de design e implementação de projetos em ANSI C baseado no uso do microcontrolador ATMega328P, evidenciando como a linguagem facilita o desenvolvimento de *software* estruturado, a portabilidade do código e o gerenciamento de tarefas complexas. O relatório segue uma estrutura formal, com fundamentação teórica, materiais e métodos, resultados e conclusões, apoiando-se em fontes consolidadas como o livro "AVR e Arduino Técnicas de Projeto" e os *datasheets* dos fabricantes, indispensáveis para o desenvolvimento de sistemas embarcados.

2. FUNDAMENTAÇÃO TEÓRICA

2.1 Microcontroladores e a Arquitetura AVR

Microcontroladores são sistemas computacionais completos em um único chip, contendo CPU, memória (RAM, Flash, EEPROM) e periféricos de Entrada/Saída (I/O), projetados para controlar tarefas específicas em sistemas embarcados. Neste trabalho, focamos na arquitetura AVR, amplamente utilizada em projetos didáticos e profissionais.

Para a descrição e o desenvolvimento dos projetos apresentados neste relatório, optou-se pelo estudo e utilização do microcontrolador ATmega328P. Este microcontrolador foi escolhido por apresentar as principais características desejáveis para o aprendizado e desenvolvimento de técnicas de projeto com sistemas embarcados.

O ATmega328P, baseado na arquitetura RISC da Atmel (atual Microchip), possui diversas funcionalidades essenciais (LIMA; VILLAÇA, 2012):

- 23 pinos de entradas e saídas (I/Os) programáveis.
- Temporizadores/Contadores (TCs) de 8 bits e 16 bits com prescalers e modos de comparação.
- Canais PWM.
- Canais de Conversão Analógico-Digital (ADC).
- Memórias Flash, SRAM e EEPROM.

2.2 ATmega328P

O ATmega328P é um microcontrolador megaAVR1 de baixa potência, baseado na arquitetura RISC, com 32 *kbytes* de memória flash, 1 *kbyte* de EEPROM e 2 *kbytes* de SRAM, operando até 20 MHz, sendo o núcleo da popular plataforma Arduino Uno. A Plataforma Arduino Uno é um sistema de prototipagem eletrônica open-source, conhecido por sua facilidade de uso e acessibilidade, que utiliza o ATmega328P e oferece diversos recursos, como pinos de Entrada/Saída (I/O) programáveis organizados em PORTB, PORTC e PORTD, com resistores de *pull-up* internos selecionáveis; controle de configuração de pinagens por meio dos registradores DDRx, PORTx e PINx; conexão USB para alimentação, comunicação serial e gravação direta; soquetes para conectar *shields* (módulos auxiliares); alimentação por USB, fonte externa ou bateria; além de uma IDE gratuita e de fácil utilização para programação, útil na prototipagem apesar de suas limitações para o desenvolvimento profissional.

2.3 Programação em Linguagem C

A linguagem C é amplamente utilizada no desenvolvimento de sistemas embarcados por oferecer um bom equilíbrio entre controle de *hardware* e facilidade de programação. No caso do ATmega328P, programar em C permite acessar diretamente os registradores do microcontrolador, configurar periféricos e tratar interrupções, sem a complexidade da linguagem *Assembly*. Com o compilador AVR-GCC, é possível escrever código eficiente e portátil, mantendo o desempenho próximo ao máximo que o *hardware* pode oferecer.

No ATmega328P, os registradores de controle como DDRx, PORTx e PINx são manipulados diretamente em C por meio de macros e operações *bit a bit*. A linguagem também permite configurar e controlar temporizadores, ADC, PWM, USART e outras interfaces com comandos simples, o que agiliza o desenvolvimento e reduz o número de erros. O tratamento de interrupções é feito por meio de rotinas ISR(), que substituem a necessidade de manipulação manual da tabela de vetores.

Além disso, a linguagem C oferece estruturação clara do código com o uso de funções, tipos personalizados (*typedef*, *struct*), e organização em arquivos, o que favorece a manutenção e reuso. Isso torna a linguagem ideal para projetos de médio e grande porte, onde clareza, modularidade e depuração são essenciais.

Mesmo não oferecendo o mesmo nível de controle de baixo nível que o *Assembly*, C permite otimizações importantes e pode ser combinado com trechos em *Assembly* quando necessário. Com ferramentas como o AVR Studio ou Microchip Studio, o desenvolvedor pode escrever, simular, depurar e programar o microcontrolador com integração total ao ambiente de desenvolvimento.

2.4 Portas de Entrada/Saída (I/O) do ATmega328P

Os pinos de entrada e saída (I/O) do ATmega328P são organizados nos grupos PORTB, PORTC e PORTD. Cada grupo funciona como uma porta de 8 bits (exceto o PORTC, que tem 7 pinos). Cada pino pode ser configurado como entrada ou saída e possui um resistor interno de *pull-up* que pode ser ativado. Os pinos conseguem fornecer ou absorver corrente suficiente para acionar LEDs, mas é importante respeitar os limites de corrente para evitar danos.

O controle dos pinos é feito por três registradores: DDRx define se o pino é entrada (0) ou saída (1); PORTx escreve valores no pino ou ativa o *pull-up* quando em modo entrada; e PINx lê o estado atual do pino. Os pinos podem ser manipulados individualmente sem afetar os outros do mesmo grupo, usando instruções como SBI e CBI. Também é possível acessar os registradores por diferentes faixas de endereço, dependendo da instrução usada (*IN/OUT* ou *LD/ST*).

Além do uso como I/O digital, muitos pinos também têm funções alternativas, como entrada analógica (ADC), comunicação serial (*USART*, SPI, I2C), saídas PWM ou interrupções externas. No Arduino Uno, por exemplo, o pino digital 13 (PB5) já vem ligado a um LED. Pinos como PD0 e PD1 são usados para comunicação USB e precisam ter a *USART* desativada se forem usados como I/O comum em *Assembly*. É recomendável configurar os pinos não utilizados com *pull-up* ativo para evitar consumo de corrente desnecessário.

2.5 Matriz de LED e o Circuito Integrado MAX7219

Uma matriz de LED é uma grade organizada de diodos emissores de luz (LEDs), geralmente dispostos em linhas e colunas — como, por exemplo, uma matriz 8×8, que contém 64 LEDs. Esse arranjo permite o controle individual de cada ponto luminoso através de técnicas de multiplexação, reduzindo a quantidade de pinos necessária para acionar todos os LEDs. Em sistemas embarcados, as matrizes de LED são utilizadas para formar caracteres, símbolos ou padrões gráficos simples, funcionando como uma pequena tela de saída visual. O controle eficiente da matriz exige atualização contínua para manter a exibição estável, o que pode ser feito por *hardware* externo ou por rotinas dedicadas no microcontrolador.

Nesse sentido, o MAX7219 se apresenta como uma excelente ferramenta auxiliar dado que é um circuito integrado voltado para o controle de matrizes de LED e *displays* de sete segmentos. Ele permite acionar todos os 64 LEDs da matriz com apenas 3 linhas de comunicação com o microcontrolador (usando o protocolo SPI simplificado). O chip gerencia internamente a multiplexação, o brilho dos LEDs (via controle de intensidade) e a atualização dos dados, aliviando significativamente a carga de processamento do sistema principal. Com ele, é possível enviar os dados de exibição por meio de registradores internos, permitindo ao microcontrolador se concentrar em outras tarefas. O MAX7219 é amplamente usado em aplicações que exigem visualização dinâmica e compacta, como jogos, relógios e interfaces gráficas simples.

2.6 Leitura de Botões/Teclas

Ler botões é uma tarefa básica ao usar um microcontrolador como o ATmega328P. Para isso, basta configurar o pino como entrada ($DDRx = 0$) e ler seu estado pelo registrador $PINx$. É comum ativar o resistor de *pull-up* interno ($PORTx = 1$) para garantir um nível lógico definido quando o botão não está pressionado.

Um problema comum é o “*bounce*”, que gera sinais instáveis durante o acionamento do botão. Para evitar leituras erradas, usa-se “*debounce*” por *software*, com pequenos atrasos (cerca de 10 ms) após o pressionamento ou soltura do botão. Também é possível usar interrupções para detectar eventos, mas o bounce pode gerar interrupções indesejadas, exigindo cuidado extra.

Quando há várias teclas, usa-se um teclado matricial, que organiza os botões em linhas e colunas, economizando pinos. O microcontrolador ativa uma linha por vez e verifica quais colunas estão ativas, identificando a tecla pressionada. É importante usar resistores de *pull-up* nas colunas para garantir leitura estável. Outra técnica é usar o ADC com divisores de tensão: cada tecla aplica uma voltagem diferente ao pino analógico, e o microcontrolador identifica qual foi pressionada com base nesse valor. Ambas as soluções são eficientes, e a escolha depende do número de teclas e recursos disponíveis no projeto.

2.7 Joystick

O *joystick* analógico é um dispositivo de entrada que permite detectar movimentos em dois eixos — horizontal (X) e vertical (Y) — por meio de dois potenciômetros internos. Cada eixo é acoplado a um cursor que desliza sobre uma resistência variável, gerando uma tensão analógica proporcional à posição do botão. Quando o *joystick* está na posição central, os sinais dos eixos geralmente se estabilizam próximos a 2,5V, representando a posição neutra. Ao movimentar o botão para qualquer direção, a tensão em um dos eixos aumenta ou diminui, permitindo ao microcontrolador identificar o sentido e intensidade do movimento. Essa leitura é feita por meio de conversão analógica-digital (ADC), o que permite utilizar o *joystick* como uma interface simples e intuitiva para controle direcional em aplicações embarcadas, como jogos, menus ou robótica.

2.8 Temporizadores/Contadores (TCs)

O ATmega328P possui três temporizadores/contadores: TC0 e TC2 (ambos de 8 *bits*) e TC1 (de 16 bits). Eles servem para contar pulsos do *clock* e gerar eventos em momentos exatos, como temporizações, sinais PWM ou interrupções. Um evento importante é o estouro, que ocorre quando o contador atinge seu valor máximo e volta para zero. Os temporizadores podem trabalhar em modos diferentes, como o modo normal (contagem contínua), modo CTC (zera ao atingir um valor específico) e modos PWM (usados para controlar a largura de pulsos). Para controlar sua velocidade, cada TC pode usar um “*prescaler*” — um divisor de clock que torna a contagem mais lenta e permite temporizações mais longas.

O TC1, por ser de 16 bits, é o mais preciso e permite medições mais detalhadas, como medir a largura do pulso do sensor ultrassônico HC-SR04 usando o recurso de captura de entrada (*Input Capture*). O TC2 tem uma função extra: pode funcionar com um cristal externo de baixa frequência para contagem de tempo real, mas isso não é viável no Arduino Uno padrão. Esses temporizadores são muito úteis em várias tarefas: acionar LEDs com PWM, gerar som, fazer a multiplexação de *displays* ou criar atrasos e interrupções com precisão.

2.9 Memória EEPROM

A EEPROM do ATmega328P é uma memória não volátil de 1 KB usada para armazenar dados que devem ser mantidos mesmo após desligar o microcontrolador, como configurações, senhas ou medições. Ela permite leitura e escrita de dados *byte a byte* por meio de registradores específicos (EEAR, EEDR e EECR). A leitura é rápida, mas a escrita é lenta (cerca de 3,3 ms) e tem um limite de 100.000 ciclos por *byte*.

Para escrever ou ler na EEPROM manualmente, é preciso seguir uma sequência de passos para evitar erros. A escrita exige cuidado extra para não causar desgaste desnecessário, pois cada operação de gravação gasta um pouco da vida útil da célula de memória.

3. MATERIAIS E MÉTODOS

3.1 Projeto 04 - Controle de LED RGB

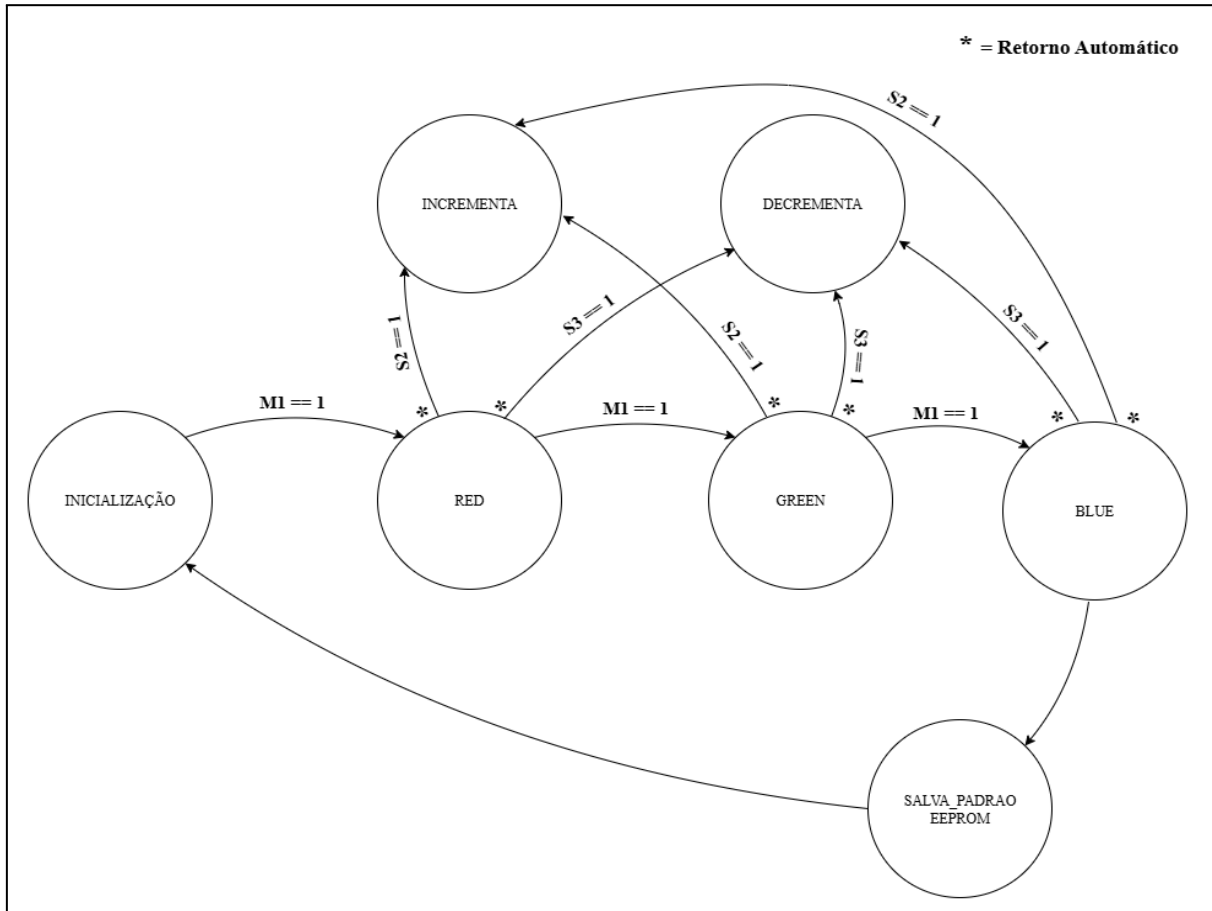
O projeto 04 tem como finalidade o desenvolvimento de um *firmware*, escrito em linguagem ANSI C, para um sistema embarcado baseado no microcontrolador AVR ATmega328P, cujo objetivo é controlar um LED RGB por meio de uma Interface Homem-Máquina (IHM).

A IHM do sistema é composta por três *pushbuttons* (M, ▲ e ▼), um *display* LCD e o próprio LED RGB. Ao ser ligado, o sistema exibe no *display* os valores iniciais de cada uma das três cores, todos iniciando em zero. A navegação entre os canais de cor é feita pelo botão M, que move o cursor (representado por um asterisco "*") para o lado do valor que se deseja ajustar. O ciclo segue a sequência: *RED* → *GREEN* → *BLUE* → nenhum. Cada novo toque em M desloca o cursor para o próximo campo, e ao final do ciclo, ele desaparece, sinalizando que nenhuma cor está selecionada no momento.

Uma vez que uma cor esteja selecionada, o usuário pode utilizar os botões ▲ e ▼ para aumentar ou diminuir seu valor, respectivamente, dentro do intervalo de 0 a 255. Com isso, a resposta do LED deve ser imediata: conforme o valor da cor é alterado, ele reflete visualmente a nova composição de cor em tempo real. Essa dinâmica proporciona ao usuário da IHM controle direto dos níveis de vermelho, verde e azul de forma visual e interativa.

Por fim, o sistema também implementa uma lógica de aceleração no ajuste dos valores: caso o botão ▲ ou ▼ permaneça pressionado por mais de 5 segundos, o incremento ou decremento acelera, permitindo alterações rápidas sem perder precisão nos ajustes mais finos. Ao final do processo de ajuste, os valores configurados de cada cor são salvos em memória não volátil (EEPROM), assegurando que a configuração escolhida seja mantida mesmo após desligamentos ou reinicializações do sistema. É possível verificar a descrição do funcionamento da IHM implementada a partir da **Figura 01**, a máquina de estados do *firmware*.

Figura 01 - Máquina de estados para o algoritmo de funcionamento do Controle de LED RGB.



Fonte: Autoria Própria.

3.1.1. Correções do Projeto 04

Visando a implementação do firmware descrito no tópico anterior, o projeto foi segmentado em torno de interrupções, controle por PWM, e atualização otimizada da interface LCD. Para dar início ao programa, o microcontrolador é configurado para trabalhar com entradas digitais nos pinos PC1, PC2 e PC3, correspondendo aos botões M, ▲ e ▼. A função nomeada “buttons_init()” é responsável por configurar esses pinos como entradas com *pull-ups* internos ativados.

O controle da cor do LED RGB é feito por PWM via timers internos, com isso a função “pin_PWM_RGB()” inicializa os temporizadores *TIMER1* e *TIMER2* em modo *Fast PWM*, com os pinos de saída configurados para controlar os canais vermelho, verde e azul do LED RGB. O uso dos temporizadores especificados anteriormente justifica-se quanto ao controle dos canais vermelho e verde realizado pelo *TIMER1*, com 16 bits. Adicionalmente, o *TIMER2* é responsável pelo controle do canal azul dado que se trata de um temporizador de 8

bits. Os valores PWM são ajustados pela função “set_color_RGB()”, que atribui os valores de intensidade diretamente aos registradores correspondentes, alterando o brilho de cada cor do LED. Dessa forma, os valores atribuídos a cada cor são atualizados dinamicamente, permitindo que a composição de cores do LED RGB seja refletida em tempo real, à medida que os ajustes são feitos.

A interface com o *display* LCD é implementada em modo 4 bits, utilizando um conjunto de rotinas que cuidam da inicialização do periférico, posicionamento do cursor, escrita de caracteres e atualização de valores numéricos. Para otimizar a exibição, a lógica de atualização compara os valores atuais com os anteriores, atualizando apenas os campos modificados. Isso evita reescritas desnecessárias e melhora a fluidez da interface.

As interações do usuário com os botões são gerenciadas por meio de interrupções externas. Cada evento de pressão nos botões é capturado por uma rotina de interrupção, que interpreta se a ação corresponde à navegação entre canais de cor ou à solicitação de ajuste. Quando o botão de incremento ou decremento permanece pressionado por tempo prolongado, um temporizador adicional é ativado, e o sistema altera automaticamente o passo de ajuste, promovendo uma variação mais rápida e prática do valor.

A lógica central do sistema é conduzida por uma função responsável por interpretar o estado atual dos botões e do seletor de cor. Essa função garante que os valores permaneçam dentro dos limites válidos (entre 0 e 255) e atualiza imediatamente o LED RGB com os novos parâmetros. Todo esse ciclo ocorre dentro do *loop* principal do programa, que também mantém o *display* sincronizado com os dados mais recentes, proporcionando uma experiência contínua e reativa ao usuário. A figura 02 apresenta o registro do funcionamento da IHM desenvolvida para controle do LED RGB.

Figura 02 - Funcionamento do Controle de LED RGB.



Fonte: Autoria Própria.

3.2 Projeto 05 - Jogo *Snake* Simplificado

O Projeto 05 tem como objetivo implementar uma versão simplificada do clássico jogo *Snake* em um sistema embarcado baseado no microcontrolador AVR Atmega328P. O controle do jogo é realizado por meio de um *joystick* analógico de dois eixos, enquanto a visualização da cobra ocorre em uma matriz de LEDs 8x8, controlada pelo circuito integrado MAX7219. O firmware foi desenvolvido em linguagem C padrão ANSI, levando em conta as limitações do *hardware* e priorizando a fidelidade à dinâmica original do jogo.

No início, a cobra é posicionada centralmente na matriz, com três segmentos, e se move continuamente para a direita. O jogador pode alterar a direção da cobra a qualquer momento utilizando o *joystick*, sendo que o sistema implementa uma zona morta para evitar mudanças acidentais de direção devido a ruídos no sinal analógico. O crescimento da cobra ocorre em intervalos regulares, tornando o jogo progressivamente mais desafiador, e a velocidade de movimentação aumenta de forma gradual conforme o comprimento da cobra cresce. O jogo é reiniciado automaticamente em caso de colisão com as bordas da matriz ou

com o próprio corpo da cobra, sendo exibida uma animação visual de "game over" na matriz de LEDs.

Durante a execução, as posições X e Y dos segmentos da cobra são exibidas em tempo real no *display* LCD, auxiliando no acompanhamento do estado interno do jogo e facilitando o processo de depuração. O projeto demonstra a integração eficiente entre múltiplos periféricos (ADC, SPI, *display* LCD e matriz de LEDs), além de explorar conceitos de lógica de jogo, controle de tempo e manipulação de entradas analógicas em sistemas embarcados.

3.2.1 Correções do Projeto 05

O Projeto 05 foi implementado com sucesso, atingindo o objetivo de reproduzir o jogo *Snake* em um ambiente embarcado com o microcontrolador AVR Atmega328P. O sistema apresentou funcionamento estável, com resposta adequada ao controle via *joystick* analógico e visualização clara da movimentação da cobra na matriz de LEDs 8x8 controlada pelo MAX7219. A utilização do *display* LCD para exibir as posições dos segmentos da cobra facilitou o acompanhamento do estado do jogo e contribuiu para o processo de depuração.

Entre os pontos positivos, destacam-se a implementação de uma zona morta no processamento do *joystick*, que reduziu interferências e garantiu maior precisão no controle direcional, e a lógica de crescimento e aceleração da cobra, que tornou o jogo mais dinâmico e desafiador. A animação de "game over" na matriz de LEDs proporcionou uma transição visual interessante entre partidas, enriquecendo a experiência do usuário. No geral, o projeto cumpriu seus objetivos e demonstrou integração eficiente entre múltiplos periféricos e técnicas de programação embarcada. A Figura 03 apresenta a versão final do jogo desenvolvido.

3.3 Projeto 06 - Filtro FIR com ajuste dinâmico

O Projeto 06 consiste no desenvolvimento de um filtro digital do tipo FIR (Finite Impulse Response) implementado no microcontrolador AVR Atmega328P, utilizando a linguagem C padrão ANSI. O sistema permite o ajuste dinâmico dos coeficientes do filtro em tempo real por meio de uma Interface Homem-Máquina (IHM) composta por um *display* LCD e botões físicos. O usuário pode navegar entre os coeficientes, visualizar seus valores em ponto flutuante e alterá-los conforme necessário, proporcionando flexibilidade para testes e experimentação de diferentes respostas do filtro. O firmware também inclui funcionalidades

para salvar e carregar os coeficientes na memória EEPROM, garantindo a persistência das configurações mesmo após desligamentos. O objetivo principal do projeto é oferecer uma plataforma prática e didática para o estudo e aplicação de filtros digitais, permitindo a modificação dos parâmetros do filtro de forma intuitiva e interativa.

3.3.1 Correções do Projeto 06

O Projeto 06 foi implementado com sucesso, cumprindo o objetivo de desenvolver um filtro digital FIR com ajuste dinâmico de coeficientes em tempo real, utilizando o microcontrolador AVR ATmega328P. O sistema permite ao usuário navegar pelos coeficientes do filtro e visualizar seus valores em ponto flutuante por meio de uma interface composta por *display* LCD e botões, tornando o ajuste dos parâmetros acessível e didático.

O código apresenta uma estrutura modular, com separação clara das funções em arquivos específicos para a máquina de estados, controle do LCD, leitura dos botões, manipulação da EEPROM, processamento do filtro FIR e aquisição de dados via ADC. Entre os pontos positivos, destacam-se a implementação dos coeficientes em ponto flutuante, a interface intuitiva para navegação, a persistência dos dados dos coeficientes na EEPROM e o uso de interrupções para leitura dos botões, o que melhora a responsividade do sistema.

Como possíveis melhorias, sugere-se permitir a edição direta dos valores dos coeficientes pelo usuário, implementar *feedback* visual no LCD para indicar alterações ou operações de salvar/carregar, adicionar proteção contra valores inválidos e expandir a interface para seleção de diferentes tipos de filtros. De modo geral, o projeto atinge seus objetivos e serve como uma base sólida para o estudo prático de filtros digitais e sistemas embarcados com interação homem-máquina.

4. RESULTADOS

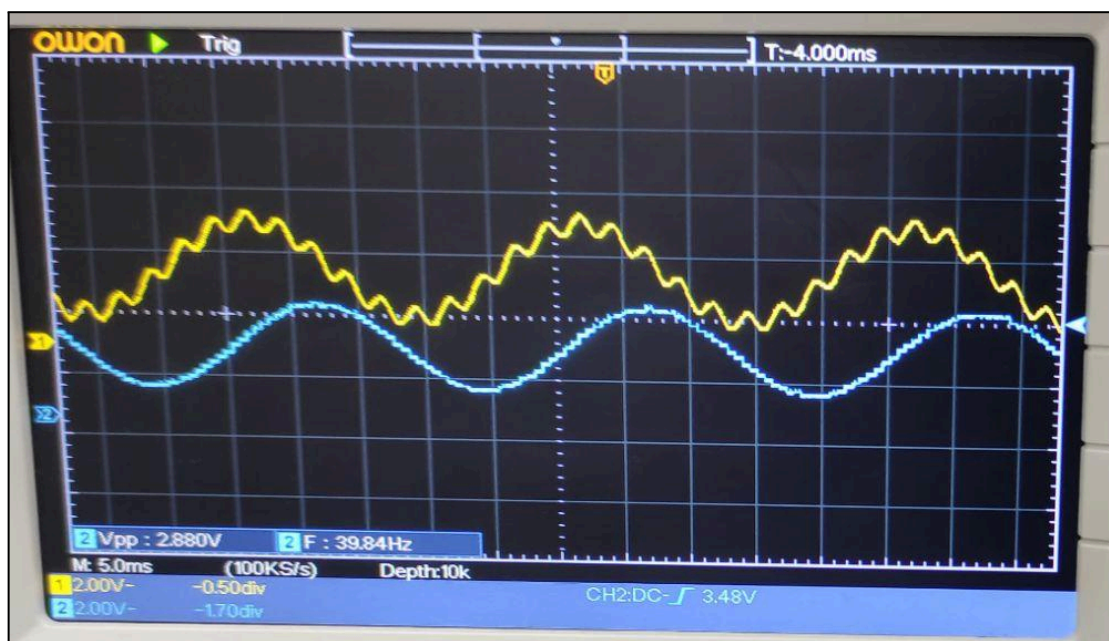
Durante a execução dos Projetos 04, 05 e 06, foi possível observar na prática o funcionamento de diferentes aplicações embarcadas utilizando o microcontrolador AVR ATmega328P, cada uma explorando aspectos distintos de interação homem-máquina, controle de periféricos e processamento de sinais. No Projeto 04, o sistema de controle do LED RGB apresentou resposta imediata e precisa aos comandos do usuário, com navegação intuitiva entre os canais de cor, ajuste eficiente dos valores de intensidade e transições suaves proporcionadas pelo controle PWM. A lógica de aceleração nos botões e a atualização

otimizada do *display* LCD. No entanto, a execução proposta nesse relatório para o projeto Controlador de LED RGB não satisfaz a condição de salvar a configuração final das cores na memória não volátil (EEPROM).

No Projeto 05, a implementação do jogo *Snake* exigiu ajustes na leitura do *joystick*, incluindo a definição de uma zona morta para evitar comandos indesejados devido a ruídos, além da calibração dos tempos de leitura do ADC e atualização dos *displays*. Essas correções foram fundamentais para garantir uma jogabilidade fluida e responsiva, permitindo ao usuário controlar a cobra de forma natural e acompanhar o estado do jogo em tempo real tanto na matriz de LEDs quanto no *display* LCD.

Já no Projeto 06, o foco esteve no desenvolvimento de um filtro digital FIR com ajuste dinâmico de coeficientes. Durante os testes, foi necessário ajustar a frequência de amostragem e o sinal de entrada para compatibilizar o funcionamento do filtro com as limitações do sistema, além de corrigir a configuração da saída analógica R2R. Também foi preciso desabilitar temporariamente a máquina de estados para garantir o processamento em tempo real do filtro. Após essas adaptações, foi possível observar o efeito do filtro FIR na saída do circuito, evidenciando a importância do ajuste fino de parâmetros e da integração eficiente entre *hardware* e *software* em sistemas embarcados, a Figura 03 apresenta o sinal de saída obtido com o filtro projetado.

Figura 03 - Sinal de entrada (amarelo) e sinal de saída (azul) do Filtro FIR projetado.



Fonte: Autoria Própria.

5. CONCLUSÃO

O desenvolvimento dos projetos apresentados neste relatório consolidou a aplicação de conceitos avançados de sistemas microcontrolados utilizando a linguagem C no microcontrolador ATmega328P. A transição do *Assembly* para o C permitiu a implementação de sistemas com maior complexidade lógica e interativa, como demonstrado com sucesso nas três frentes de trabalho.

O Projeto de Controle de LED RGB (Projeto 04) demonstrou a capacidade de criar interfaces homem-máquina (IHM) responsivas, integrando controle por PWM, leitura de botões e persistência de dados em EEPROM. Em seguida, o Jogo *Snake* (Projeto 05) aprofundou a complexidade ao gerenciar múltiplos periféricos em tempo real — um *joystick* analógico (ADC), uma matriz de LEDs (SPI) e um *display* LCD —, reforçando a importância da gestão de tempo e da lógica de jogo. Por fim, o Filtro FIR (Projeto 06) representou o maior desafio técnico, ao implementar um algoritmo de processamento digital de sinais com coeficientes dinâmicos, evidenciando a relação crítica entre o *software* e as limitações do *hardware*, especialmente no que tange à frequência de amostragem.

Em conjunto, os projetos provaram a eficácia da linguagem C para o desenvolvimento de firmware modular e funcional. A experiência prática adquirida reafirmou que, embora uma linguagem de alto nível acelere o desenvolvimento, o conhecimento profundo da arquitetura do microcontrolador e de seus periféricos continua sendo fundamental para criar soluções embarcadas robustas e eficientes.

REFERÊNCIAS BIBLIOGRÁFICAS

ATMEL. *ATmega48A/PA/88A/PA/168A/PA/328P: 8-bit AVR microcontroller with 4/8/16/32K bytes in-system programmable flash*. San Jose: Atmel Corporation, 2015. Disponível em: <https://www.microchip.com>. Acesso em: 14 maio 2025.

ATMEL. *Atmel AVR 8-bit Instruction Set Manual*. San Jose: Atmel Corporation, 2016. Disponível em: <https://www.microchip.com>. Acesso em: 14 maio 2025.

LIMA, Charles Borges de; VILLAÇA, Marco Valério Miorim. *AVR e Arduino: técnicas de projeto*. 2. ed. Florianópolis: Edição dos Autores, 2012. 632 p. ISBN 978-85-911400-1-5.

MAZIDI, Muhammad Ali; NAIMI, Sarmad; NAIMI, Sepehr. *AVR Microcontroller and Embedded Systems: Using Assembly and C*. 1. ed. Boston: Pearson Education, 2011.

ATMEL Corporation. *ATmega328P: 8-bit AVR Microcontroller with 32K Bytes In-System Programmable Flash*. Versão 7810D–AVR–01/15. San Jose: Atmel Corporation, 2015.

ANEXOS

- Link de acesso para os registros em vídeo dos projetos:
<https://drive.google.com/drive/folders/16WWT8i5NKO3rLNnILD2IUGkjqLduf7Dg?usp=sharing>
- Link para o repositório de códigos dos projetos:
<https://github.com/BigLeno/ELE3717-SM>